



## Bases de données avancées

Public : DI3-IG3

MA Amar

[medabdellahiamar@yahoo.fr](mailto:medabdellahiamar@yahoo.fr)

**AU 2023 - 2024**



# Objectif du cours

- Utiliser un SGBD professionnel largement répandu
- Maîtriser les fonctions de base du PL/SQL Oracle
- Découvrir et utiliser les outils d'administration de base de données Oracle



# Plan:

- Introduction à Oracle
- SQL pour Oracle
- PL/SQL Oracle
- Outils d'administration



# Introduction

Une base de données est une collection de données stockées dans des fichiers et accessibles à la demande pour plusieurs utilisateurs;

Une BD est faite pour enregistrer des faits, des opérations au sein d'un organisme ( banque, université, hôpital, ...)

# Introduction



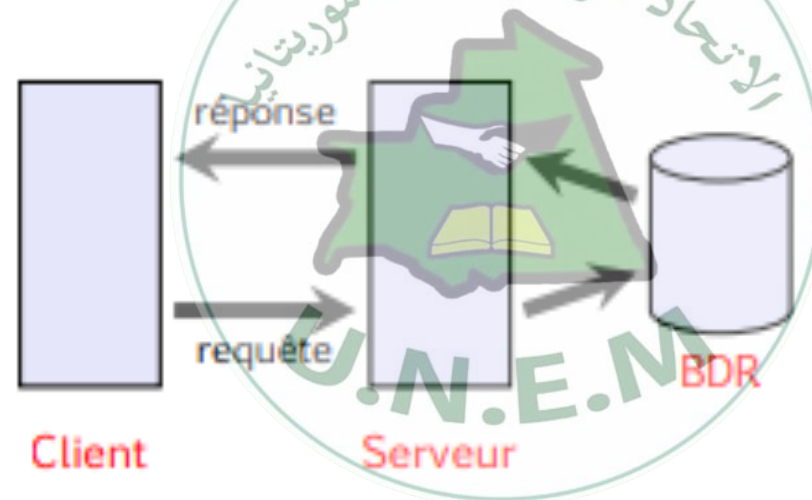
## Les bases de données relationnelles.

Schématiquement, il s'agit d'un ensemble de tables contenant des données reliées entre elles par des relations ; on y extrait de l'information par le biais de requêtes exprimées dans un langage spécifique.

# Introduction



Un système de gestion de bases de données est le logiciel qui organise et gère les données ; l'utilisateur interagit avec lui par l'intermédiaire de requêtes exprimées en SQL (Structured Query Language).  
Architecture deux-tiers



Le logiciel client de la machine de l'utilisateur envoie des requêtes au logiciel serveur installé sur la machine qui traite les requêtes



# Oracle (entreprise)

- Est une entreprise américaine créée en 1977
- Par Larry Ellison
- 2009 : Rachat de Sun Microsystems
  - JAVA
  - MYSQL
  - ...



Sun  
Microsystems  
Constructeur d'ordinateurs





# Les produits d'Oracle

- Oracle Database

- Un système de gestion de base de données



ORACLE<sup>®</sup>  
D A T A B A S E

- Oracle SQL Developer

- Un logiciel pour manumuler une base de données



- Oracle Weblogic Server

- Un serveur d'application



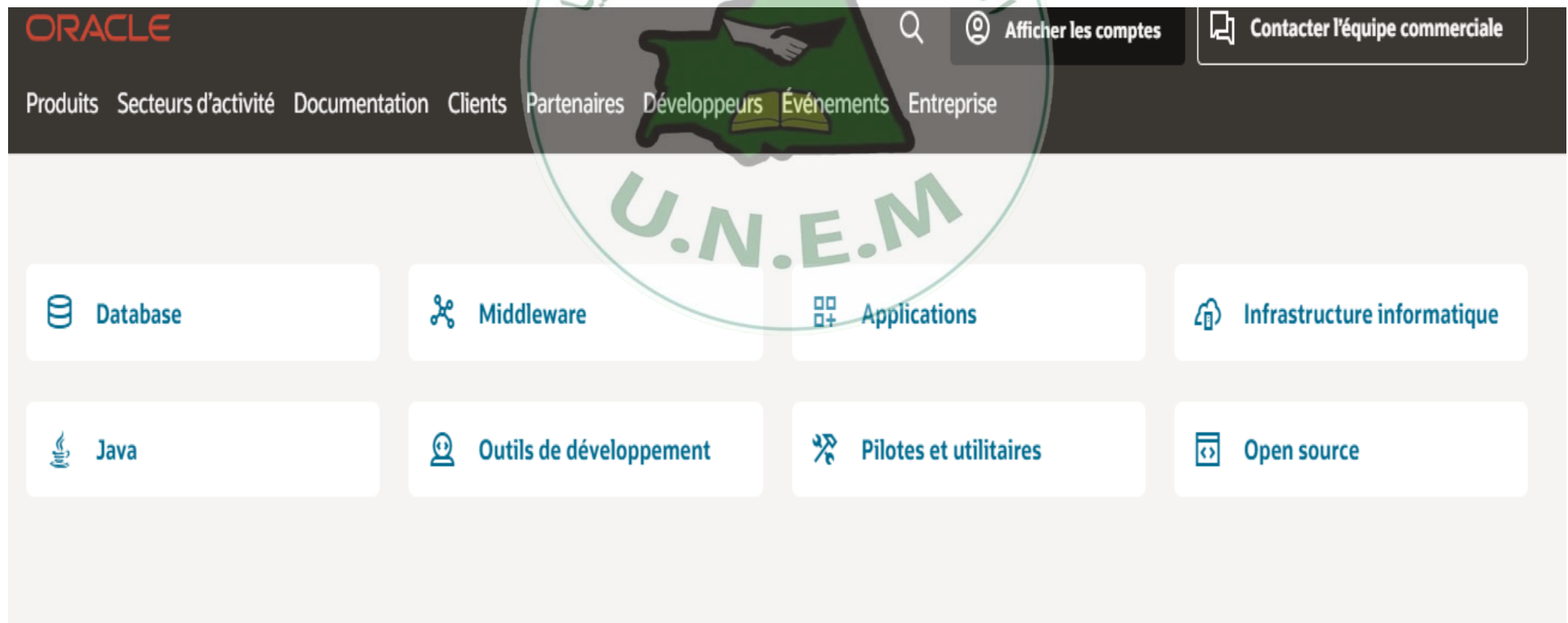
ORACLE<sup>®</sup>  
WebLogic Server





# Liste complète des produits

- Consulter
- <https://www.oracle.com/fr/downloads/>





# Pourquoi SGBD-Oracle

- ❖ **Non fermé:** tu peux connaître ce qui se passe derrière de A à Z, ce qui n'est pas possible pour SQL Server de Microsoft ou DB2 d'IBM par exemple.
- ❖ **Portable (Multiplateforme):** s'installe sur la majorité des OS:
  - Windows de Microsoft
  - AIX d'IBM
  - Solaris d'Oracle
  - UP/UX de HP
  - Linux
- ❖ **Performances, Sécurité ...**

- Il occupe la première place dans le marché des SGBDR.



# Oracle (database)

- Oracle Database est le SGBD qui permet de stocker, gérer, administrer et manipuler des données d'un grand volume tout en assurant performance, sécurité et accès concurrentiel.



# Oracle

- Évolutions du produit :
  - depuis la version 7, architecture client-serveur ;
  - depuis la version 8, le serveur est objet-relationnel ;
- Version actuelle : **21c** (13 janvier 2021)



# Versions

- Chaque version est commercialisée sous différentes éditions:
- **Editions**
  - **L'Enterprise Edition (EE)**
    - Est la version la plus complète mais aussi la plus onéreuse
  - **La Standard Edition (SE)**
    - Est une version moins complète que la version EE, (4 Processeurs au maximum)
  - **L'eXpress Edition (XE)**
    - gratuite fonctionne sur des machines à un processeur.
    - Limitation de taille

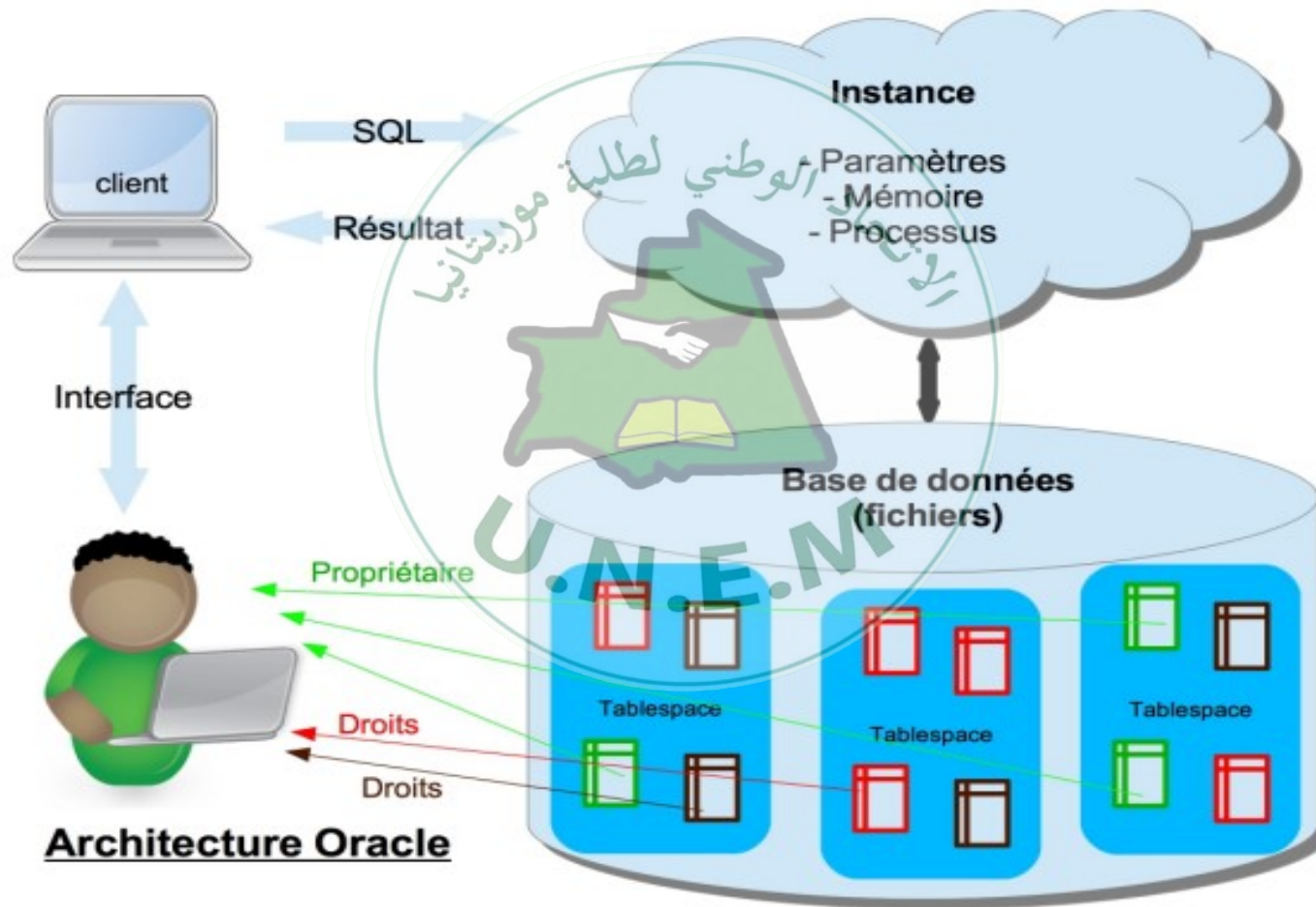


# Serveur Oracle

- Un serveur oracle , c'est le système de gestion de base de données, se compose d'une **instance oracle** et **base de donnée oracle**
  - **Instance** ensemble de processus et de zone mémoires qui permet de gérer la base de données
  - **Base de donnée** : ensemble de fichier (sur disque) contenant les données , les information sur les données, le journal de modification.



# Architecture





# Langage SQL Pour Oracle



– Introduction

– SQL comme LDD

- création du schéma (tables,...)

– SQL comme LMD

- Insert, Update, Delete
- SQL comme Langage de Requêtes
  - Requêtes de base
  - Ordonner les réponses
  - Jointures et les operateurs
  - Fonctions de groupe et regroupement de lignes
  - Sous-requêtes
  - Vues
  - Fonctions pour requêtes SQL

– SQL comme LCD (langage de contrôle des données)





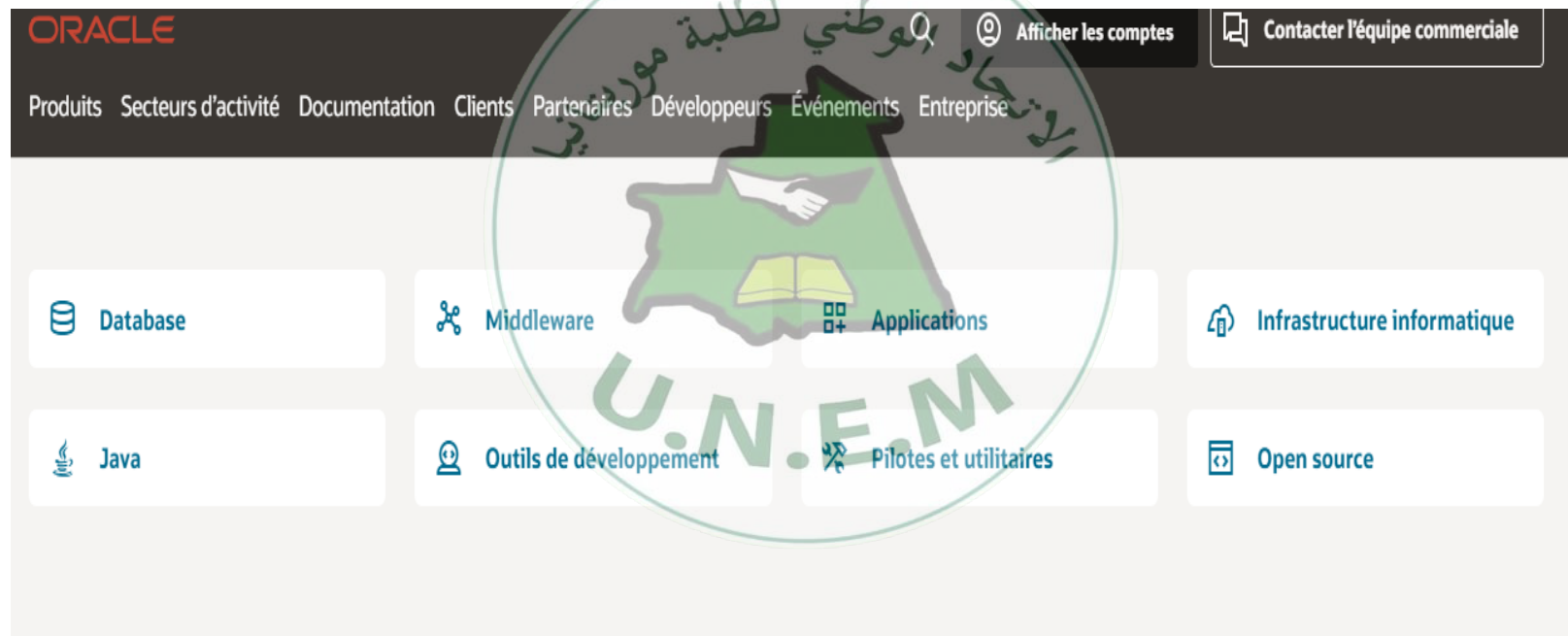
# Langage SQL

- Le **SQL** (Structured Query Language) est un langage permettant de communiquer avec une base de données.
- IL permet de :
  - définir les données (LDD)
  - interroger la base de données (Langage de requêtes)
  - manipuler les données (LMD)
  - contrôler l'accès aux données (LCD).
- Conçu par IBM dans les années 70



# SQL et Oracle

- Oracle est SGBD qui utilise SQL
- Installation (Version 21c XE)



- <https://linuxhint.com/how-to-install-oracle-21c-on-windows/>

# Premier Contact



## SQL PLUS interactif – SQL développer

SQL Plus

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mar: Oct. 17 14:10:54 2023  
Version 21.3.0.0.0  
  
Copyright (c) 1982, 2021, Oracle. All rights reserved.  
  
Entrez le nom utilisateur :
```

Oracle SQL Developer

File Edit View Navigate Run Debug Source Tools Help

Connections Reports

xe\_local BOOKS LIST\_A\_RATING

Columns Data Indexes Constraints Grants Statistics Column Statistics Triggers Dependence

Actions...

| Column Name        | Data Type       | Nulla... | Data Def... | CO... | Primary...    | COMME |
|--------------------|-----------------|----------|-------------|-------|---------------|-------|
| BOOK_ID            | VARCHAR2(20 ... | No       | (null)      | 1     | 1 (null)      |       |
| TITLE              | VARCHAR2(50 ... | No       | (null)      | 2     | (null) (null) |       |
| AUTHOR_LAST_NAME   | VARCHAR2(30 ... | No       | (null)      | 3     | (null) (null) |       |
| AUTHOR_FIRST_NA... | VARCHAR2(30 ... | Yes      | (null)      | 4     | (null) (null) |       |
| RATING             | NUMBER          | Yes      | (null)      | 5     | (null) (null) |       |

# SQL Comme LDD



- **Identificateurs :**

- lettre suivie par : lettre ou chiffre ou \_ ou # ou \$
- maximum 30 caractères
- différent d'un mot clé
  - ASSERT, ASSIGN, AUDIT, COMMENT, DATE, DECIMAL,
  - DEFINITION, FILE, FORMAT, INDEX, LIST, MODE,
  - OPTION, PARTITION, PRIVILEGE, PUBLIC, SELECT,
  - SESSION, SET, TABLE

- pas de distinction entre **majuscules** et **minuscules**

# SQL Comme LDD



## Tables :

Relations d'un schéma relationnel stockées sous **tables**

**table** : formée de lignes et de colonnes

| id | prenom | nom    | ville | age |
|----|--------|--------|-------|-----|
| 1  | Ahmed  | Dia    | NKtt  | 24  |
| 2  | Feih   | Hamadi | NDB   | 36  |
| 3  | Fatma  | Hassen | Kaédi | 27  |

# SQL Comme LDD



Table

Oracle SQL Developer

File Edit View Navigate Run Debug Source Tools Help

Connections

HR

Enter SQL Statement:

```
Select * from Departments;
```

Results

| DEPARTMENT_ID | DEPARTMENT_NAME    | MANAGER_ID | LOC |
|---------------|--------------------|------------|-----|
| 1             | 10 Administration  | 200        |     |
| 2             | 20 Marketing       | 201        |     |
| 3             | 30 Purchasing      | 114        |     |
| 4             | 40 Human Resources | 203        |     |
| 5             | 50 Shipping        | 121        |     |

All Rows Fetched: 11

Editing

# SQL Comme LDD



- **Tables : colonnes**

- toutes les données d'une colonne sont du même type
- identificateur unique pour les colonnes d'une même table
- 2 colonnes dans 2 tables différentes peuvent avoir le même nom





# Type de données

- SQL reconnaît plusieurs types de données.

## ORACLE : Types pour les chaînes de caractères

Les types CHAR et NCHAR permettent de stocker des chaînes de caractères de taille fixe

- CHAR(*taille*) ou NCHAR(*taille*)

Les types VARCHAR2 et NVARCHAR2 permettent de stocker des chaînes de caractères de taille variable

- VARCHAR(*taille\_max*)

MAIS de préférence

- VARCHAR2(*taille\_max*) ou NVARCHAR2(*taille\_max*)





# Type de données

## ORACLE : Types pour les chaînes de caractères



- **NCHAR(5)** : chaînes de 5 caractères
- **VARCHAR2(20)** : chaînes de 20 caractères au plus
- **'Administration', 'Marketing'**

Activer Wi



## Autres Types

- Les types **CLOB** et **NCLOB** permettent de stocker des flots de caractères (exemple : du texte)



# Type de données

- Le type **NUMBER** sert à stocker des entiers positifs ou négatifs, des réels à virgule fixe ou flottante.

ORACLE : Types numériques

exemples

- NUMBER** : nombre en virgule flottante avec jusqu'à 38 chiffres significatifs
- NUMBER(*nb\_chiffres*, *nb\_décimales*)** : nombre en virgule fixe



# Type de données

**DATE** : Ce type de données permet de stocker des données constituées d'une date.

La précision est composée du siècle, de l'année, du mois, du jour, de l'heure, des minutes et des secondes.

**TIMESTAMP** ; est plus précis dans la définition d'un moment



# Type de données

## Données binaires;

Les types **BLOB** et **BFILE** permettent de stocker des données non structurées comme le multimédia (images, sons, vidéo, etc.).



# Les tables



**Une table** est un ensemble de lignes et de colonnes.

La création consiste à définir le nom de ses colonnes, leur *type* et les règles de gestion s'appliquant à la colonne (CONSTRAINT).



# CREATION DES TABLES

## création de table

**CREATE TABLE** nom\_de\_table (liste de définition\_de\_colonne,  
[liste de contrainte\_de\_table]);

définition\_de\_colonne ::=

nom\_de\_colonne

(nom\_de\_domaine ou type)

[liste de contrainte\_de\_colonne]

[**DEFAULT** valeur\_par\_défaut]





# Contraintes d'intégrité

Quand on crée une table, il faut définir les **contraintes d'intégrité** que devront respecter les données à saisir



## Les contraintes

### **NOT NULL ou NULL :**

Interdit (NOT NULL) ou autorise (NULL) l'insertion de valeur NULL pour cet attribut.

### **UNIQUE :**

Deux n-uplets ne peuvent recevoir des valeurs identiques pour cet attribut, mais l'insertion de valeur NULL est toutefois autorisée.

### **PRIMARY KEY :**

Désigne l'attribut comme clé primaire de la table.



# Type de données

ORACLE : Absence de valeur

**NULL** : représente l'absence de valeur pour tous les types de données. **Ce n'est pas une valeur**

- pour les types chaîne de caractères :
  - la chaîne vide ' ' représente aussi l'absence de valeur
- pour les types numériques
  - le nombre 0 ne représente pas l'absence de valeur



# Les contraintes

## **CHECK (conditionC):**

Vérifie lors de l'insertion de n-uplets  
que l'attribut réalise la  
condition conditionC.

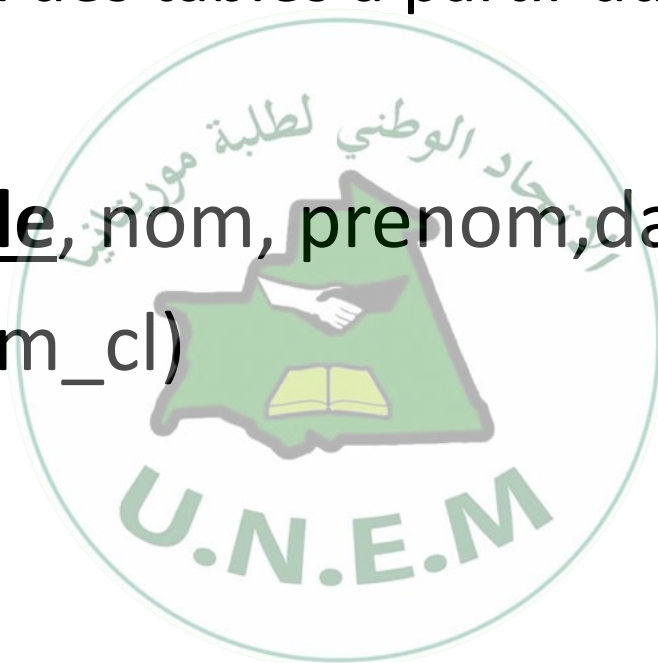


# CREATION DES TABLES

**Exemple** : Création des tables a partir du schéma :

Etudiant(Matricule, nom, prenom, datenaiss, #codeCl)

Classe(CodeC, nom\_cl)





# CREATION DES TABLES

```
CREATE TABLE Classe(CodeC number(5) CONSTRAINT Cle_P_Classe Primary Key,  
nom_cl Varchar2(40) )
```

```
CREATE TABLE etudiant(Matricule number(5) CONSTRAINT Cle_P_etudiant primary  
key,  
nom varchar2(20) NOT NULL,  
prenom varchar2(30),  
datenaiss Date,  
codeCl number(5),  
CONSTRAINT Cle_Et_Classe Foreign key(codeCl) references Classe(codeC) )
```



# CREATION DES TABLES





# CREATION DES TABLES

Soit le modèle relationnel suivant relatif à la gestion des notes annuelles d'une promotion d'étudiants **Gestion\_notes** :

ETUDIANT(NEtudiant, Nom, Prenom, Ville)

MATIERE(CodeMat, LibelleMat, CoeffMat,)

EVALUER(#NEtudiant, #CodeMat, Date, Note)

**Créer ces tables**



# Les tables



Supprimer la table etudiant

**drop table etudiant;**

Modifier le Nom d'une table

**alter table etudiant rename to etudiants;**

Modifier le type d'une colonne

**alter table etudiants MODIFY nom varchar2(50);**

# Les tables



Supprimer une colonne

**alter table étudiants drop** COLUMN **ville** ;

Ajouter une colonne

**alter table etudiants add** COLUMN **Email varchar2(30)** ;



## Création et Insertion de données

- L'insertion de données dans une table s'effectue à l'aide de la commande **INSERT INTO**. Cette commande permet au choix d'inclure une seule ligne à la base existante ou plusieurs lignes.

## Création et Insertion de données



### Insérer une ligne en spécifiant toutes les colonnes

La syntaxe pour remplir une ligne avec cette méthode est la suivante :

INSERT INTO table VALUES ('valeur 1', 'valeur 2', ...)

| <u>NEtudiant</u> | Nom | Prénom | Ville |
|------------------|-----|--------|-------|
|                  |     |        |       |
|                  |     |        |       |
|                  |     |        |       |
|                  |     |        |       |



# Table Etudiant

| <u>N</u> Etudiant | Nom      | Prénom       | Ville |
|-------------------|----------|--------------|-------|
| 1                 | Issa     | Arbi         | Nktt  |
| 2                 | Benany   | Fatma        | Kiffa |
| 3                 | Diakité  | Mohamed      | Boghé |
| 4                 | Mohamed  | Mohamed      | Nktt  |
| 5                 | Mohamed  | Yaghoub      | Ndb   |
| 6                 | Houssein | Zeinebou     | Atar  |
| 7                 | Moctar   | Marieme      | Nktt  |
| 8                 | Ahmed    | Nebgha       | Ndb   |
| 9                 | Aly      | Daouda       | Kiffa |
| 10                | Hamadi   | Abdarrahmane | Kaédi |



# Exemple

Ajouter un étudiant dans la table étudiant en respectant l'ordre des colonnes

**INSERT INTO** étudiant **VALUES** (12435, 'Aly', 'Ahmed', 'NDB') ;

| <u>NEtudiant</u> | Nom     | Prénom  | Ville |
|------------------|---------|---------|-------|
| 1                | Issa    | Arbi    | Nktt  |
| 2                | Benany  | Fatma   | Kiffa |
| 3                | Diakité | Mohamed | Boghé |



# La commande UPDATE

La commande UPDATE permet d'effectuer des modifications sur des lignes existantes.

## Syntaxe

La syntaxe basique d'une requête utilisant UPDATE est la suivante :

UPDATE table SET nom\_colonne\_1 = 'nouvelle valeur' WHERE ***condition***



1. Changer le nom de l'étudiant n° 4 par : Bakar

update etudiant set nom="Bakar" where  
netudiant=4;

2. Changer le nom NDB par Nouadhibou

update etudiant set ville="Nouadhibou" where  
ville = "NDB";

| <u>NEtudiant</u> | Nom     | Prénom  | Ville |
|------------------|---------|---------|-------|
| 1                | Issa    | Arbi    | Nktt  |
| 2                | Benany  | Fatma   | Kiffa |
| 3                | Diakité | Mohamed | Boghé |
| 4                | Mohamed | Mohamed | Nktt  |





# La commande DELETE

- La commande DELETE en SQL permet de supprimer des lignes dans une table.
- **Syntaxe**

La syntaxe pour supprimer des lignes est la suivante :

**DELETE FROM `table`  
WHERE condition**

**Attention** : s'il n'y a pas de condition WHERE alors **toutes** les lignes seront supprimées et la table sera alors vide.



Supprimer l'étudiant 3 qui a quitté l'institut

| <u>NEtudiant</u> | Nom      | Prénom       | Ville |
|------------------|----------|--------------|-------|
| 1                | Issa     | Arbi         | Nktt  |
| 2                | Benany   | Fatma        | Kiffa |
| 3                | Diakité  | Mohamed      | Boghé |
| 4                | Mohamed  | Mohamed      | Nktt  |
| 5                | Mohamed  | Yaghoub      | Ndb   |
| 6                | Houssein | Zeinebou     | Atar  |
| 7                | Moctar   | Marieme      | Nktt  |
| 8                | Ahmed    | Nebgha       | Ndb   |
| 9                | Aly      | Daouda       | Kiffa |
| 10               | Hamadi   | Abdarrahmane | Kaédi |



## b- Les requetés de bases



# Les requêtes de bases



## Extraction de données:

A partir d'une seule table

Select A1, A2,...An

From

T

Where Condition ;

2eme

*3eme, la clause SELECT nous demande les attributs a garder dans le resultat*

*1ere, la clause FROM demande la table en entree*



## Example 1

Compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

# Les requêtes de bases



**Selection: Example :** toutes les lignes et toutes les colonnes

Compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

Select \* From Compte ;

output

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

# Les requêtes de bases



## Selection: Example 1

Compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

Select \* From Compte  
Where montant<1000;

montant < 1000?  
oui!

output

| num | transactionID | Date     | montant |
|-----|---------------|----------|---------|
| 102 | 1             | 10/22/09 | 500.000 |

# Les requêtes de bases



## Example 1 cont.

compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

Select \* from compte  
Where montant<1000;

montant < 1000?  
oui!

output

| num | transactionID | Date     | montant |
|-----|---------------|----------|---------|
| 102 | 1             | 10/22/09 | 500.000 |
| 102 | 2             | 10/29/09 | 200.000 |



# Les requêtes de bases



## Example 1 cont.

compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

montant < 1000?  
NON!

Select \* from compte  
Where montant < 1000;

A ignorer

output

| num | <u>transactionID</u> | Date     | montant |
|-----|----------------------|----------|---------|
| 102 | 1                    | 10/22/09 | 500.000 |
| 102 | 2                    | 10/29/09 | 200.000 |

# Les requêtes de bases



## Example 1.

compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 4                    | 10/29/09 | 1000.000  |
| 105 | 5                    | 11/2/09  | 10000.000 |

Select \* from compte  
Where montant<1000;

montant < 1000?  
NON!

A ignorer

output

| num | <u>transactionID</u> | Date     | montant |
|-----|----------------------|----------|---------|
| 102 | 1                    | 10/22/09 | 500.000 |
| 102 | 2                    | 10/29/09 | 200.000 |

# Les requêtes de bases



## Selection: Example 2

compte

| num | <u>transactionID</u> | Date     | montant   |
|-----|----------------------|----------|-----------|
| 102 | 1                    | 10/22/09 | 500.000   |
| 102 | 2                    | 10/29/09 | 200.000   |
| 104 | 3                    | 10/29/09 | 1000.000  |
| 105 | 4                    | 11/2/09  | 10000.000 |

Select **num**, **montant** from compte  
Where montant<1000;

montant < 1000?  
YES!

| num | transactionID | Date     | montant |
|-----|---------------|----------|---------|
| 102 | 1             | 10/22/09 | 500.000 |
| 102 | 2             | 10/29/09 | 200.000 |

Resultat intermediaire

WHERE

SELECT

| num | montant |
|-----|---------|
| 102 | 500.000 |
| 102 | 200.000 |

Resultat final

# Les requêtes de bases



## Selection: Example 3

**compte**

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | checking |
| 102        | Khaled       | 2000.00 | checking |
| 103        | Azeiz        | 500.00  | savings  |
| 104        | Moussa       | 1200.00 | checking |
| 105        | Sultan       | 8000.00 | checking |

```
SELECT *  
FROM compte  
WHERE Type = "checking" AND proprietaire="Moussa";
```

*Que retourne cette requete?*

# Les requêtes de bases



## Example 3 cont.

compte

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | checking |
| 102        | Khaled       | 2000.00 | checking |
| 103        | Azeiz        | 500.00  | savings  |
| 104        | Moussa       | 1200.00 | checking |
| 105        | Sultan       | 8000.00 | checking |

```
SELECT *  
FROM compte  
WHERE Type = "checking" AND proprietaire="Moussa";
```

*resultat:*

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | Checking |
| 104        | Moussa       | 1200.00 | checking |

# Les requêtes de bases



## Selection: Example 4

Compte

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | checking |
| 102        | Khaled       | 2000.00 | checking |
| 103        | Azeiz        | 500.00  | savings  |
| 104        | Moussa       | 1200.00 | checking |
| 105        | Sultan       | 8000.00 | checking |

```
SELECT proprietaire, Type
FROM compte
WHERE Type= "checking" AND solde<=400;
```

resultat

| proprietaire | Type |
|--------------|------|
|              |      |

# Les requêtes de bases



## Selection: Example 5

Compte

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | checking |
| 102        | Khaled       | 2000.00 | checking |
| 103        | Azeiz        | 500.00  | savings  |
| 104        | Moussa       | 1200.00 | checking |
| 105        | Sultan       | 8000.00 | checking |

```
SELECT proprietaire, Type
FROM Compte
WHERE proprietaire= "Moussa" OR solde<=600;
```

*Resultat?*

# Les requêtes de bases



## Example 5 cont.

Compte

| <u>num</u> | proprietaire | solde   | Type     |
|------------|--------------|---------|----------|
| 101        | Moussa       | 700.00  | checking |
| 102        | Khaled       | 2000.00 | checking |
| 103        | Azeiz        | 500.00  | savings  |
| 104        | Moussa       | 1200.00 | checking |
| 105        | Sultan       | 8000.00 | checking |

```
SELECT proprietaire, Type
FROM Compte
WHERE proprietaire= "Moussa" OR
      Balance<=600;
```

| <b>Owner</b> | <b>Type</b> |
|--------------|-------------|
| Moussa       | checking    |
| Azeiz        | savings     |
| Moussa       | checking    |





# Table Etudiant

| <u>NEtudiant</u> | Nom      | Prénom       | Ville |
|------------------|----------|--------------|-------|
| 1                | Issa     | Arbi         | Nktt  |
| 2                | Benany   | Fatma        | Kiffa |
| 3                | Diakité  | Mohamed      | Boghé |
| 4                | Mohamed  | Mohamed      | Nktt  |
| 5                | Mohamed  | Yaghoub      | Ndb   |
| 6                | Houssein | Zeinebou     | Atar  |
| 7                | Moctar   | Marieme      | Nktt  |
| 8                | Ahmed    | Nebgha       | Ndb   |
| 9                | Aly      | Daouda       | Kiffa |
| 10               | Hamadi   | Abdarrahmane | Kaédi |



## Exemple sur la table etudiant

1. Afficher la Liste des étudiants
2. Quel est le nombre total d'étudiants ?
3. Quels sont les étudiants qui habitent NKTT ou Kiffa ?
4. On souhaite obtenir uniquement les étudiants dont le nom commence par "M".
5. Quels sont les étudiants qui n'habitent pas NKTT ?
6. Quelles sont, parmi l'ensemble des notes, la note la plus haute et la note la plus basse ?



Opérateurs

# Exemple, sur la base ETUDIANTS



ETUDIANT (NumEtu, Nom, Prenom, DateNaiss, Ville)

EPREUVE (CodeEpr, DateEpr, Lieu)

PASSER (#NumEtu, #CodeEpr, Note)

# ETUDIANT



| Num | Nom | Prénom |  |
|-----|-----|--------|--|
|-----|-----|--------|--|

|      |        |           |
|------|--------|-----------|
| 1110 | Dupont | Albertine |
| 2002 | West   | James     |

## PASSER (table « pont »)

| NumEtu# | CodeEpr# | Note |
|---------|----------|------|
| 1110    | INFOS101 | 15,5 |
| 2002    | ECOS101  | 8,5  |
| 2002    | ECOS102  | 13   |
| 1110    | GESS201  | 14   |
| 2002    | GESS201  | 14,5 |

# EPREUVE

| CodeEpr  | DateEpr    | Lieu   |
|----------|------------|--------|
| ECOS101  | 15/01/2016 | Aubrac |
| ECOS102  | 16/01/2016 | Aubrac |
| GESS201  | 25/05/2016 | D201   |
| INFOS101 | 20/01/2016 | D101   |



# Etoile et champ calculé

- ▶ Tous les n-uplets d'une table : étoile ( \* )

ex. `SELECT * FROM Etudiant`

- ▶ Champs calculés

ex. Transformation de notes sur 20 en notes sur 10

`SELECT Note / 2 FROM Passer`



## Opérateurs de Comparaison

| Opérateur            | Signification                                   |
|----------------------|-------------------------------------------------|
| BETWEEN<br>...AND... | Compris entre ... et ...<br>(bornes comprises)  |
| IN (liste)           | Correspond à une valeur de la liste             |
| LIKE                 | Ressemblance partielle de chaînes de caractères |
| IS NULL              | Correspond à une valeur NULL                    |

| Opérateur | Signification       |
|-----------|---------------------|
| =         | Egal à              |
| >         | Supérieur à         |
| >=        | Supérieur ou égal à |
| <         | Inférieur à         |
| <=        | Inférieur ou égal à |
| <>        | Différent de        |





# Opérateurs Logiques

| Opérateur | Signification                                                         |
|-----------|-----------------------------------------------------------------------|
| AND       | Retourne TRUE si <i>les deux</i> conditions sont <b>VRAIES</b>        |
| OR        | Retourne TRUE si <i>l'une au moins</i> des conditions est VRAIE       |
| NOT       | Ramène la valeur TRUE si la condition qui suit l'opérateur est FAUSSE |





## Règles de Priorité

| Ordre de priorité | Opérateur                          |
|-------------------|------------------------------------|
| 1                 | Tous les opérateurs de comparaison |
| 2                 | NOT                                |
| 3                 | AND                                |
| 4                 | OR                                 |

**Les parenthèses permettent de modifier les règles de priorité**

# ETUDIANT



| Num | Nom | Prénom |  |
|-----|-----|--------|--|
|-----|-----|--------|--|

|      |        |           |
|------|--------|-----------|
| 1110 | Dupont | Albertine |
| 2002 | West   | James     |

## PASSER (table « pont »)

| NumEtu# | CodeEpr# | Note |
|---------|----------|------|
| 1110    | INFOS101 | 15,5 |
| 2002    | ECOS101  | 8,5  |
| 2002    | ECOS102  | 13   |
| 1110    | GESS201  | 14   |
| 2002    | GESS201  | 14,5 |

# EPREUVE

| CodeEpr  | DateEpr    | Lieu   |
|----------|------------|--------|
| ECOS101  | 15/01/2016 | Aubrac |
| ECOS102  | 16/01/2016 | Aubrac |
| GESS201  | 25/05/2016 | D201   |
| INFOS101 | 20/01/2016 | D101   |

# Operateurs de restriction



Épreuves se déroulant après le 01/01/2016

ex. Notes comprises entre 10 et 20

ex. Notes indéterminées (sans valeur)



NumEtu# Nom Prénom

//////////

1110 Dupont Albertine

2002 West James

PASSER (table « pont »)

## EPREUVE

CodeEpr DateEpr Lieu

ECOS101 15/01/2016 Aubrac

ECOS102 16/01/2016 Aubrac

GESS201 25/05/2016 D201

INFOS101 20/01/2016 D101

NumEtu# CodeEpr# Note

1110 INFOS101 15,5

2002 ECOS101 8,5

2002 ECOS102 13

1110 GESS201 14

2002 GESS201 14,5

# Operateurs de restriction



ex. Etudiant·es habitant une ville dont le nom se termine  
par sur-Saône





# ETUDIANT



| Num | Nom | Prénom |  |
|-----|-----|--------|--|
|-----|-----|--------|--|

|      |        |           |
|------|--------|-----------|
| 1110 | Dupont | Albertine |
| 2002 | West   | James     |

## PASSER (table « pont »)

| NumEtu# | CodeEpr# | Note |
|---------|----------|------|
| 1110    | INFOS101 | 15,5 |
| 2002    | ECOS101  | 8,5  |
| 2002    | ECOS102  | 13   |
| 1110    | GESS201  | 14   |
| 2002    | GESS201  | 14,5 |

# EPREUVE

| CodeEpr  | DateEpr    | Lieu   |
|----------|------------|--------|
| ECOS101  | 15/01/2016 | Aubrac |
| ECOS102  | 16/01/2016 | Aubrac |
| GESS201  | 25/05/2016 | D201   |
| INFOS101 | 20/01/2016 | D101   |

# Operateurs de restriction



ex. Prénoms des étudiant·es dont le nom est Dupont, Durand ou Martin



# ETUDIANT



| Num | Nom | Prénom |  |
|-----|-----|--------|--|
|-----|-----|--------|--|

|      |        |           |
|------|--------|-----------|
| 1110 | Dupont | Albertine |
| 2002 | West   | James     |

## PASSER (table « pont »)

| NumEtu# | CodeEpr# | Note |
|---------|----------|------|
| 1110    | INFOS101 | 15,5 |
| 2002    | ECOS101  | 8,5  |
| 2002    | ECOS102  | 13   |
| 1110    | GESS201  | 14   |
| 2002    | GESS201  | 14,5 |

# EPREUVE

| CodeEpr  | DateEpr    | Lieu   |
|----------|------------|--------|
| ECOS101  | 15/01/2016 | Aubrac |
| ECOS102  | 16/01/2016 | Aubrac |
| GESS201  | 25/05/2016 | D201   |
| INFOS101 | 20/01/2016 | D101   |



# Operateurs logiques



- ▶ LI. ex. Épreuves se déroulant le 15/01/2016 en salle D201

```
SELECT * FROM Epreuve  
WHERE DateEpr = '15-01-2016' AND Lieu = 'D201'
```

- ▶ OU. ex. Étudiant·es né·es avant 1990 ou habitant hors Lyon

```
SELECT * FROM Etudiant  
WHERE DateNaiss < '01-01-1990' OR Ville <> 'Lyon'
```

- ▶ Combinaisons. ex. Étudiant·es né·es après 1990 et habitant Lyon ou Vienne

```
SELECT * FROM Etudiant  
WHERE DateNaiss > '31-12-1990'  
AND (Ville = 'Lyon' OR Ville = 'Vienne')
```

# ETUDIANT



| Num | Nom | Prénom |  |
|-----|-----|--------|--|
|-----|-----|--------|--|

|      |        |           |
|------|--------|-----------|
| 1110 | Dupont | Albertine |
| 2002 | West   | James     |

## PASSER (table « pont »)

| NumEtu# | CodeEpr# | Note |
|---------|----------|------|
| 1110    | INFOS101 | 15,5 |
| 2002    | ECOS101  | 8,5  |
| 2002    | ECOS102  | 13   |
| 1110    | GESS201  | 14   |
| 2002    | GESS201  | 14,5 |

# EPREUVE

| CodeEpr  | DateEpr    | Lieu   |
|----------|------------|--------|
| ECOS101  | 15/01/2016 | Aubrac |
| ECOS102  | 16/01/2016 | Aubrac |
| GESS201  | 25/05/2016 | D201   |
| INFOS101 | 20/01/2016 | D101   |



# Clause ORDER BY

- Tri des lignes avec la clause ORDER BY
  - ASC : ordre croissant (par défaut)
  - DESC : ordre décroissant
- La clause ORDER BY se place à la fin de l'ordre SELECT

## ► Tri du résultat

ex. Par ordre alphabétique [inverse] de nom

```
SELECT * FROM Etudiant
```

```
ORDER BY Nom [DESC]
```